Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЕТ
к лабораторной работе №4
на тему:
«ЗАЩИТА ОТ АТАКИ ПРИ УСТАНОВКЕ TCP-СОЕДИНЕНИЯ И ПРОТОКОЛОВ ПРИКЛАДНОГО УРОВНЯ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артём Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Краткие теоритичесие сведения 4
3 Выполнение работы 5
Вывод 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Реализовать сервер, защищённый от атаки при установке TCP соединения и в рамках заданного протокола прикладного уровня.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

В процессе эксплуатации сетевых сервисов, основанных на протоколе TCP, возможны различные виды атак, направленных на нарушение доступности сервера или исчерпание его ресурсов. К таким атакам относятся: отправка сообщений, превышающих допустимый размер, что может вызвать переполнение буфера; медленная отправка данных, когда соединение устанавливается, но передача ведется с большой задержкой, удерживая ресурсы сервера; флуд-атаки, при которых с одного IP-адреса поступает чрезмерное количество запросов за короткий промежуток времени; а также установка пустых соединений без передачи данных, что приводит к исчерпанию лимита на количество одновременных подключений.

Для защиты от угроз применяются следующие методы: ограничение максимального размера принимаемого сообщения, установка таймаута на чтение данных, ограничение частоты запросов с одного IP-адреса с последующей временной блокировкой при превышении лимита.

3 ВЫПОЛНЕНИЕ РАБОТЫ

Сервер был развернут на локальном хосте (127.0.0.1) на порту 8080. В конфигурационном файле заданы следующие параметры безопасности: максимальный размер сообщения – 1024 байта, таймаут ожидания данных – 5 секунд, максимальное количество запросов с одного IP – 10 за 60 секунд, длительность блокировки при превышении лимита – 60 секунд. Для управления доступом был разработан класс SecurityManager, который ведет историю запросов по каждому IP, проверяет их количество в скользящем временном окне и при необходимости накладывает временную блокировку. При обработке клиентского подключения сервер сначала проверяет, не заблокирован ли IP, затем ожидает поступление данных в течение заданного таймаута, после чего контролирует размер полученного сообщения. В случае успеха отправляется ответ.

Для проверки работоспособности защитных механизмов был написан клиент, реализующий серию тестов. На рисунке 3.1 представлен результат работы со стороны клиента.

```
[NORMAL] Ответ: Echo: Hello!

--- Тест: Превышение размера данных ---
[OVERFLOW] Ответ: Error: Message too large

--- Тест: Медленная отправка ---
[SLOW] Отправляем по одному символу слова 'Hi'...
[SLOW] Ответ: Echo: H


--- Тест: Пустое соединение (занятие порта) ---
[EMPTY] Подключились...
[EMPTY] Сервер закрыл соединение по таймауту

--- Тест: Flood (лимит 10 запросов) ---
[FLOOD] Запрос №1
[FLOOD 1] Ответ: Echo: Ping
[FLOOD] Запрос №2
[FLOOD 2] Ответ: Echo: Ping
[FLOOD] Запрос №3
[FLOOD 3] Ответ: Echo: Ping
[FLOOD] Запрос №4
[FLOOD 4] Ответ: Echo: Ping
[FLOOD] Запрос №5
[FLOOD 5] Ответ: Echo: Ping
[FLOOD] Запрос №6
[FLOOD 6] Ответ: Echo: Ping
[FLOOD] Запрос №7
[FLOOD 7] Ошибка: [Errno 104] Connection reset by peer
[FLOOD] Запрос №8
[FLOOD 8] Ошибка: [Errno 104] Connection reset by peer
```

Рисунок 3.1 – Результат работы программы со стороны клиента

В тесте на превышение размера данных клиент отправил сообщение длиной 1025 байт. Сервер, обнаружив, что размер превышает лимит, вернул ошибку «Error: Message too large», что отражено в выводе клиента.

При проверке защиты от медленной отправки клиент сначала передал один символ «H», после чего выдержал паузу, превышающую таймаут соединения, и лишь затем отправил второй символ. Как видно из логов сервера, после истечения таймаута соединение было закрыто. Клиент при попытке отправить второй символ получил ошибку, что подтверждает разрыв соединения сервером.

Тест с пустым соединением заключался в установке TCP-соединения без отправки данных. Сервер через 5 секунд закрыл его по таймауту.

Наиболее показательным стал тест на флуд. Клиент последовательно отправлял запросы с одного IP. Первые 10 запросов были успешно обработаны – на сервере видны соответствующие записи о получении данных, а клиент получал. Последующие запросы отклонялись с отказом в обслуживании, и клиент получал ошибку. Ниже, также, приведёт рисунок 3.2 и со стороны сервера.

```
[SERVER] Запущен на 127.0.0.1:8080
[*] Новое соединение: 127.0.0.1:35042
[DATA] Получено от 127.0.0.1: Hello!
[*] Новое соединение: 127.0.0.1:35056
[!] Слишком большой объем данных от 127.0.0.1
[*] Новое соединение: 127.0.0.1:35060
[DATA] Получено от 127.0.0.1: H
[*] Новое соединение: 127.0.0.1:53802
[!] Таймаут соединения для 127.0.0.1 (возможная атака)
[*] Новое соединение: 127.0.0.1:53816
[DATA] Получено от 127.0.0.1: Ping
[*] Новое соединение: 127.0.0.1:53822
[DATA] Получено от 127.0.0.1: Ping
[*] Новое соединение: 127.0.0.1:53836
[DATA] Получено от 127.0.0.1: Ping
[*] Новое соединение: 127.0.0.1:54744
[DATA] Получено от 127.0.0.1: Ping
[*] Новое соединение: 127.0.0.1:54746
[DATA] Получено от 127.0.0.1: Ping
[*] Новое соединение: 127.0.0.1:54756
[DATA] Получено от 127.0.0.1: Ping
[*] Новое соединение: 127.0.0.1:54766
[SECURITY] IP 127.0.0.1 превысил лимит и забанен на 60 сек.
[!] Отказ в обслуживании для 127.0.0.1
[*] Новое соединение: 127.0.0.1:54772
[SECURITY] IP 127.0.0.1 заблокирован. Осталось: 59 сек.
[!] Отказ в обслуживании для 127.0.0.1
[*] Новое соединение: 127.0.0.1:54776
[SECURITY] IP 127.0.0.1 заблокирован. Осталось: 59 сек.
[!] Отказ в обслуживании для 127.0.0.1
```

Рисунок 3.2 – Результат работы программы со стороны сервера

ВЫВОД

В ходе выполнения лабораторной работы был разработан TCP-сервер, устойчивый к основным видам атак на прикладном уровне: превышению размера сообщения, медленной отправке, пустым соединениям и флуду. Использование асинхронного подхода позволило управлять множеством подключений, а встроенные механизмы контроля (таймаут, лимит на количество запросов, блокировка клиента) обеспечили защиту от исчерпания ресурсов. Результаты тестирования полностью подтвердили корректность работы защитных функций.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация config.py

```
# Настройки сети
HOST = '127.0.0.1'
PORT = 8080

# Лимиты безопасности
MAX_MESSAGE_SIZE = 1024
CONNECTION_TIMEOUT = 5.0
MAX_REQUESTS_PER_IP = 10
RATE_LIMIT_WINDOW = 60
BAN_DURATION = 60
```

Листинг А.2 – Реализация security.py

```
import time
from collections import defaultdict
import config


class SecurityManager:
    def __init__(self):
        self.history = defaultdict(list)
        self.bans = {}

    def is_allowed(self, ip):
        now = time.time()

        if ip in self.bans:
            if now < self.bans[ip]:
                remaining = int(self.bans[ip] - now)
                print(f"[SECURITY] IP {ip} заблокирован. Осталось: {remaining}
сек.")
                return False
            else:
                print(f"[SECURITY] Срок бана для {ip} истек. Доступ разрешен.")
                del self.bans[ip]

        self.history[ip] = [t for t in self.history[ip] if now - t <
config.RATE_LIMIT_WINDOW]

        if len(self.history[ip]) >= config.MAX_REQUESTS_PER_IP:
            self.bans[ip] = now + config.BAN_DURATION
            print(f"[SECURITY] IP {ip} превысил лимит и забанен на
{config.BAN_DURATION} сек.")
            return False

        self.history[ip].append(now)
        return True

    def validate_message_size(self, data):
        return len(data) <= config.MAX_MESSAGE_SIZE


security_mgr = SecurityManager()
```

Листинг А.3 – Реализация server.py

```
import asyncio
import config
from security import security_mgr

async def handle_client(reader, writer):
    ip, port = writer.get_extra_info('peername')
    print(f"[*] Новое соединение: {ip}:{port}")

    if not security_mgr.is_allowed(ip):
        print(f"[!] Отказ в обслуживании для {ip}")
        writer.close()
        return

    try:
        data = await asyncio.wait_for(
            reader.read(config.MAX_MESSAGE_SIZE + 1),
            timeout=config.CONNECTION_TIMEOUT
        )

        if not security_mgr.validate_message_size(data):
            print(f"[!] Слишком большой объем данных от {ip}")
            writer.write(b"Error: Message too large\n")
            await writer.drain()
        else:
            message = data.decode().strip()
            print(f"[DATA] Получено от {ip}: {message}")
            writer.write(f"Echo: {message}\n".encode())
            await writer.drain()

    except asyncio.TimeoutError:
        print(f"[!] Таймаут соединения для {ip} (возможная атака)")
    except Exception as e:
        print(f"[!] Ошибка при обработке {ip}: {e}")
    finally:
        writer.close()
        await writer.wait_closed()

async def main():
    server = await asyncio.start_server(handle_client, config.HOST,
config.PORT)
    print(f"[SERVER] Запущен на {config.HOST}:{config.PORT}")
    async with server:
        await server.serve_forever()

if __name__ == "__main__":
    asyncio.run(main())
```

Листинг А.4 – Реализация client.py

```
import socket
import time
import config


def send_request(msg, label="[TEST]"):
    try:
        with socket.create_connection((config.HOST, config.PORT), timeout=3)
as sock:
            sock.sendall(msg.encode())
            response = sock.recv(1024)
```

```
            print(f"{label} Ответ: {response.decode().strip()}")
    except Exception as e:
        print(f"{label} Ошибка: {e}")


def test_message_overflow():
    print("\n--- Тест: Превышение размера данных ---")
    big_data = "A" * (config.MAX_MESSAGE_SIZE + 1)
    send_request(big_data, label="[OVERFLOW]")


def test_slow_attack():
    print("\n--- Тест: Медленная отправка ---")
    try:
        with socket.create_connection((config.HOST, config.PORT)) as sock:
            print("[SLOW] Отправляем по одному символу слова 'Hi'...")
            sock.send(b"H")
            time.sleep(config.CONNECTION_TIMEOUT + 1
            sock.send(b"i")
            resp = sock.recv(1024)
            print(f"[SLOW] Ответ: {resp.decode()}")
    except Exception as e:
        print(f"[SLOW] Соединение разорвано сервером (как и ожидалось): {e}")


def test_rate_limiting():
    print(f"\n--- Тест: Flood (лимит {config.MAX_REQUESTS_PER_IP} запросов) --
-")
    for i in range(config.MAX_REQUESTS_PER_IP + 2):
        print(f"[FLOOD] Запрос №{i + 1}")
        send_request("Ping", label=f"[FLOOD {i + 1}]")
        time.sleep(0.1)


def test_empty_connection():
    print("\n--- Тест: Пустое соединение (занятие порта) ---")
    try:
        with socket.create_connection((config.HOST, config.PORT)) as sock:
            print("[EMPTY] Подключились...")
            time.sleep(config.CONNECTION_TIMEOUT + 1)
            data = sock.recv(1024)
            if not data:
                print("[EMPTY] Сервер закрыл соединение по таймауту")
    except Exception as e:
        print(f"[EMPTY] Ошибка: {e}")


if __name__ == "__main__":
    send_request("Hello!", label="[NORMAL]")

    test_message_overflow()

    test_slow_attack()

    test_empty_connection()

    test_rate_limiting()
```